BİL 455/533 – Parallel Processing

Exercise #2

1. **Multi-threaded 2D convolution.**
    - Assume that there are only four possible filter sizes: 3x3, 7x7, 11x11, 21x21.
    - Your input pixels will be 8-bit unsigned integers.
    - Your filter coefficients will be 32-bit floats.
    - All computations will be done in 32-bit floating point precision.
    - The output pixels will be converted back to 8-bit unsigned integers.
    - You can use the OpenCV's C++ API to read-in images. and access pixel values.
    - You can use the provided image lena_grayscale_hq.jpg.
    - You can use arbitrary center weighted filter kernels, such as Gaussian.

    i. Write a single-threaded C++ code that performs 2D convolution on 8-bit grayscale images with arbitrary 2D filters.
    ii. Write a multi-threaded C++ code that performs 2D convolution on 8-bit grayscale images with arbitrary 2D filters. Use the standard C++ threads.
    iii. Compare the execution times of the codes in parts (i) and (ii) for varying numbers of threads, for example, 4-threads, 8-threads, maximum number of threads your system allows.
    iv. Compare the execution times of the codes in parts (i) and (ii) for varying image sizes.

2. **Multi-threaded matrix multiplication.**
    - Your input matrices will be 32-bit integers -or- 32-bit floats.
    - All computations will be done in 32-bit floating point precision.
    - You can create random matrices as input.
    - You should use some linear algebra library to do sanity check.

    i. Write a single-threaded C++ code that performs matrix multiplication.
    ii. Write a multi-threaded C++ code that performs matrix multiplication. Use the standard C++ threads.
    iii. Compare the execution times of the codes in parts (i) and (ii) for varying numbers of threads, for example, 4-threads, 8-threads, maximum number of threads your system allows.
    iv. Compare the execution times of the codes in parts (i) and (ii) for varying matrix sizes.